

Mercado Fácil — supermercado on-line com Flask

O Projeto didático completo demonstra uma aplicação Flask organizada no padrão MVC, com autenticação por sessão, middleware `before\_request`, SQLite, templates Jinja2 e exportação para Excel.

Funcionalidades

- login, logout e cadastro de usuários com senha criptografada;
- middleware que protege todas as páginas privadas;
- CRUD completo de produtos, pesquisa e controle de estoque;
- carrinho de compras armazenado na sessão;
- finalização transacional, com baixa automática do estoque;
- data e hora automáticas registradas pelo SQLite;
- histórico, detalhes e impressão de comprovante;
- exportação de produtos e compras para `.xlsx`;
- layout responsivo para computador, tablet e celular;
- testes automatizados do fluxo principal.

Organização MVC

supermercado_online/

└─ app.py	# fábrica da aplicação e middleware
└─ config.py	# configurações
└─ database.py	# conexão e esquema SQLite
└─ utils.py	# moeda e filtros Jinja2
└─ controllers/	# Controllers (rotas/Blueprints)
└─ models/	# Models (consultas e regras de dados)
└─ templates/	# Views Jinja2

- └─ static/style.css # interface responsiva
- └─ instance/ # banco local criado automaticamente
- └─ exports/ # reservado para exportações locais
- └─ tests/ # testes de integração

No Flask, os templates representam a camada View. Os Blueprints em `controllers/` recebem as requisições e coordenam os Models. Os arquivos em `models/` concentram acesso ao banco e regras de persistência.

Instalação no Linux com venv

Requer Python 3.10 ou mais recente. No Ubuntu/Debian, instale o suporte a ambientes virtuais caso ainda não esteja disponível:

Entre na pasta do projeto e crie o ambiente:

```
mkdir supermercado_online
cd supermercado_online
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Execute a aplicação:

```
python app.py
```

Abra `http://127.0.0.1:5000` no navegador.

O banco `instance/supermercado.db` e todas as tabelas são criados automaticamente no primeiro início. O acesso inicial é:

- usuário: `admin`

- senha: `admin123`

Cadastre uma conta própria e troque a chave da sessão antes de publicar a aplicação no GitHub. Para sair do ambiente virtual, execute `deactivate`.

Executar os testes

Com o ambiente virtual ativo:

```
pytest -q
```

Os testes usam bancos temporários e não alteram o banco utilizado em aula.

Configuração segura

Para uma aula local, a configuração padrão funciona sem ajustes. Em uma implantação real, defina uma chave longa e aleatória:

```
export SECRET_KEY="uma-chave-longa-aleatoria-e-secreta"
```

```
python app.py
```

O servidor iniciado por `python app.py` é voltado ao desenvolvimento e não deve ser exposto diretamente na internet.

Publicar o código no GitHub

Crie primeiro um repositório vazio no GitHub. Depois, dentro da pasta do projeto:

```
git init
```

```
git add .
```

```
git commit -m "Projeto inicial do supermercado online"
```

```
git branch -M main
```

```
git remote add origin https://github.com/SEU_USUARIO/supermercado_online.git
```

```
git push -u origin main
```

O `.gitignore` impede o envio do ambiente virtual, banco local, caches, segredos e planilhas exportadas.

Observações

Valores monetários são armazenados em centavos (inteiros), evitando erros de arredondamento de `float`. Exclusão, logout e finalização usam POST, para impedir alterações acidentais por simples abertura de links. As exportações são geradas em memória, evitando conflito entre usuários simultâneos.